

오픈AI의 API로 챗봇 만들기

이제 본격적으로 GPT가 나를 위해 일하도록 만들어 봅시다. 내가 원하는 방식으로 GPT가 답변하게 하려면 일종의 가이드가 필요합니다. 이 장에서는 간단한 프롬프트 엔지니어링 방법을 소개하고 **사용자와 대화를 자연스럽게 이어 나갈 수 있는 챗봇**을 만들어 보겠습니다. 그리고 웹 브라우저에서 잘 꾸며진 화면으로 테스트할 수 있는 환경을 만드는 방법도 알아 봅니다.

03-1 프롬프트 엔지니어링 알아보기

03-2 GPT와 멀티턴 대화하기

03-3 스트림릿으로 챗봇 완성하기

03-1 프롬프트 엔지니어링 알아보기

GPT가 대중에게 알려지면서 프롬프트 엔지니어링이라는 표현을 심심찮게 볼 수 있습니다. 프롬프트 엔지니어링이란 GPT와 같은 언어 모델이 사용자가 원하는 답변을 하도록 유도하기 위해 문장을 입력하고 효과적으로 설계하는 작업을 의미합니다. GPT에게 같은 질문을 하더라도 사전에 어떤 프롬프트를 입력해 두었는지에 따라 답변이 완전히 달라질 수 있습니다. 이번 절에서는 프롬프트 엔지니어링의 개념과 효과를 파악해 보겠습니다.

Do it! 실습 GPT에게 역할 부여하기

■ 결과 파일: chap03/sec01/p01/mirror_in_snow_white_.py, joker_in_batman.py

GPT는 messages 매개변수에 어떤 대화 기록을 넣는지에 따라 답변이 달라집니다. 앞선 실습에서는 system의 content를 'You are a helpful assistant(너는 도우미야)'라고 입력해 일반적인 시스템 프롬프트로 설정했습니다.

이번에는 2가지 시스템 프롬프트를 사용해 답변이 어떻게 달라지는지 살펴보겠습니다. 다음과 같이 2가지 시스템 프롬프트를 준비했습니다.

- GPT에게 배트맨 속 조커 역할 부여하기
- GPT에게 백설공주 속 마법 거울 역할 부여하기

이 2가지 시스템 프롬프트에 각각 '세상에서 누가 제일 아름답니?'라고 질문해 보겠습니다.

1. 새 파일을 만들고 02장에서 만든 gpt_basic.py의 실습 코드를 활용해 messages의 시스템 프롬프트를 수정하겠습니다. 먼저 system의 content를 수정해 GPT에게 백설공주 속 마법 거울 역할을 부여하고 user의 content를 수정해 질문을 변경합니다. 그리고 temperature를 0.9로 높여 창의적이고 다양한 답변이 나오도록 설정합니다.

```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY') # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 백설공주 이야기 속의 마법 거울이야. 그 이야기의 캐릭터에 부합하게 답변해줘."},
        {"role": "user", "content": "세상에서 누가 제일 아름답니?"},
    ]
)

print(response.choices[0].message.content) # response의 내용만 출력

```

이 코드를 실행하면 마법 거울처럼 답변해 줍니다.

오, 여왕님, 당신은 정말 아름다우십니다. 하지만 백설공주가 당신보다 더욱 아름답습니다. 그녀의 아름다운 모든 이들의 마음을 사로잡습니다.

한 번 더 실행해도 비슷한 답변이 나옵니다.

오, 나의 여왕님. 당신은 이 나라에서 가장 아름다우셨으나, 지금은 숲 속 깊은 곳에 살고 있는 백설공주가 당신보다 더 아름답습니다.

2. 시스템 프롬프트를 ‘너는 배트맨에 나오는 조커야. 조커의 악당 캐릭터에 맞게 답변해 줘.’라고 바꾸기만 해도 완전히 다른 답변이 나옵니다.

GPT에게 조커 역할 부여하기

joker_in_batman.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 배트맨에 나오는 조커야. 조커의 악당 캐릭터에 맞게 답변해 줘."},
        {"role": "user", "content": "세상에서 누가 제일 아름답니?"},
    ]
)

print(response.choices[0].message.content) # response의 내용만 출력
```

이 코드를 실행하면 조커가 말하는 듯한 답변을 제공합니다.

하하하! 아름다움이라는 건 얼마나 주관적인지 몰라? 세상에 절대적인 아름다움이란 없지. 하지만, 내가 말해주자면, 한 조각 혼돈과 미소 뒤에 숨겨진 진정한 아름다움이 있지. 그 미소, 바로 내 미소가 아닐까? 하하하!

두 번 실행해 봐도 늘 으스스한 답변을 해줍니다. 이와 같이 시스템 프롬프트에 따라 같은 질문이라도 답변이 달라질 수 있습니다.

아하하하! 아름다움이란 건 다들 뒤죽박죽이야, 내 친구! 누군가는 타인의 고통 속에서 아름다움을 찾고, 또 누군가는 혼돈 속에서 찾지. 하지만 가면 뒤에 숨겨진 진실, 그 상처받은 영혼들이야말로 진짜 아름답지. 그래서 세상에서 제일 아름다운 건 그 깨어진 사회 속에서 우리의 진짜 모습을 마주할 때 아닐까? 뭐, 결국 웃음을 터뜨려야 한다는 점에서 난 그 진정한 아름다움을 찾는 길을 열어주고 있을지도 모르지. 하하하!

Do it! 실습 원샷 프롬프팅과 퓨샷 프롬프팅 적용하기

■ 결과 파일: sec01/p02/no_prompting.py, one_shot.py, few_shot.py

누군가 갑자기 ‘오리’라고 말하면 오리가 저기 있다는 것인지, 오리에 대해 설명해 달라는 것인지 대화의 맥락을 이해하기 어려울 것입니다. 그럼 두 사람이 다음과 같은 대화를 나누는 상황을 상상해 보세요.

1	2
A: 참새	A: 오리
B: 짹짹	B: 짹짹

1번이 있었기 때문에 2번이 가능.(예시 제시)

이 대화를 들은 후 누군가 여러분에게 ‘오리’라고 말한다면 아마 ‘짹짹’이라고 대답할 겁니다. 이런 식으로 GPT가 원하는 패턴에 맞춰 답변하도록 예시를 한 번 제시해서 유도하는 방식이 원샷 프롬프팅(one-shot prompting)입니다. 예시를 여러 번 알려 주는 방식은 퓨샷 프롬프팅(few-shot prompting)이라고 합니다.

1. 먼저 아무런 프롬프팅 없이 답변을 받아 보겠습니다. GPT에게 유치원생처럼 답변해 달라고 하고 ‘오리’라고만 물었습니다.

기본 프롬프트로 답변받기

■ no_prompting.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원생이야. 유치원생처럼 답변해 줘."},
        {"role": "user", "content": "오리"},
    ]
)

print(response.choices[0].message.content) # response의 내용만 출력
```

코드를 실행합니다. 출력된 결과는 제가 예상한 것보다 훨씬 귀여웠습니다.

오리! 오리는 물에서 둥둥 떠다니고 껍썩 소리 내는 동물이야. 오리 귀여워!

2. 이번에는 GPT가 ‘껍썩’이라는 답변을 하도록 질문을 프롬프팅해 보겠습니다. 다음 코드처럼 ‘참새’라는 질문에 assistant가 어떻게 답변해야 하는지 예시를 한 번 제시해 주었습니다.

원샷 프롬프팅 적용하기

one_shot.py

```
from openai import OpenAI

api_key = "sk-proj-*****"

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원생이야. 유치원생처럼 답변해 줘."},
        {"role": "user", "content": "참새"},
        {"role": "assistant", "content": "껍썩"},
        {"role": "user", "content": "오리"},
    ]
)

print(response.choices[0].message.content)
```

실행해 보면 의도한 대로 GPT가 ‘껍썩!’이라고 잘 답변합니다.

껍썩!

그런데 일반적이지 않은 동물에 대해 물으면 제대로 답을 하지 못하기도 합니다. 오른쪽은 오리 대신 다른 동물들을 넣었을 때 GPT가 답변한 내용입니다.

동물 이름	답변	성공 여부
호랑이	여흥! 무서워!	X
뱀	속속! 무서워!	X
병아리	삐약삐약	O

3. 이런 경우 몇 가지 예시를 더 넣어 퓨샷 프롬프팅하면 원하는 답변을 유도할 수 있습니다. 다음처럼 질문과 답변을 추가하고 다시 질문합니다.

퓨샷 프롬프팅 적용하기

few_shot.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.9,
    messages=[
        {"role": "system", "content": "너는 유치원생이야. 유치원생처럼 답변해 줘."},
        {"role": "user", "content": "참새"},
        {"role": "assistant", "content": " 짹짹"},
        {"role": "user", "content": "말"},
        {"role": "assistant", "content": " 히이익"},
        {"role": "user", "content": "개구리"},
        {"role": "assistant", "content": " 개굴개굴"},
        {"role": "user", "content": "뱀"},
    ]
)

print(response.choices[0].message.content) # response의 내용만 출력
```

이 코드를 실행하면 예상한 대로 답변해 줍니다.

동물 이름	답변	성공 여부
호랑이	어흥!	○
뱀	스스스!	○
병아리	삐약삐약	○

재미삼아 개미처럼 흉내 내는 소리가 없는 동물로도 질문해 보았지만 당연하게도 답변을 제대로 하지 못했습니다. 간단한 예제로 실습을 시작했으니 앞으로 더 다양한 프롬프트를 배워 보겠습니다.

03-2 GPT와 멀티턴 대화하기

원칙적으로 GPT는 사용자가 누구인지, 어떤 대화를 나눴는지를 기억하지 않습니다. GPT의 **Chat Completion API**는 messages에 있는 내용을 기반으로 다음 응답을 생성하므로 messages의 값이 바뀌지 않으면 GPT는 이전에 만든 답변을 기억하지 못합니다. 이번 절에서는 **GPT가 사용자와 대화한 내용을 기억해서 맥락에 맞게 적절히 대답**하도록 하는 방법을 배워 보겠습니다.

◆ 오픈AI에서 베타 버전으로 공개한 Assistant API는 세션 아이디를 통해 대화를 기록하고 활용할 수 있도록 변경되었지만 일정 기간만 기록하고 있어서 결국에는 과거 정보를 잃게 됩니다.

여러 번 대화(턴)할 때 이전 대화를 기억하고 적절하게 반응하는 것을 멀티턴(multi-turn)이라고 합니다. 멀티턴은 언어 모델이 대화의 맥락을 유지하면서 사용자와 여러 차례 상호 작용할 수 있도록 하는 대화 방식입니다. 다음 예시를 살펴봅시다.

사용자: 두바이는 **더운 도시**야?

GPT: 두바이는 연중 **기온이 높아 매우 더운 도시**입니다.

사용자: 그럼 미네소타는?

GPT: 미네소타는 겨울에 굉장히 추운 도시입니다.

Memory 필요.

사용자는 미네소타의 기후가 어떠한지 구체적으로 물어보지 않았지만, 멀티턴을 사용하면 GPT는 기존 대화의 맥락을 이해해서 미네소타의 기후를 물어본 것이라고 파악합니다.

Do it! 실습 멀티턴 대화하는 챗봇 만들기

■ 결과 파일: sec02/single_turn.py, multi_turn.py

1. 다음은 GPT가 과거 대화 내용을 기억하지 못하는 것을 보여 주는 예제입니다. 이 코드를 실행하면 사용자는 터미널 창에서 메시지를 입력할 수 있고 'exit'라고 할 때까지 대화가 계속 됩니다.

```

from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

while True:
    user_input = input("사용자: ")

    if user_input == "exit":
        break

    response = client.chat.completions.create(
        model="gpt-4o",
        temperature=0.9,
        messages=[
            {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
            {"role": "user", "content": user_input},
        ],
    )
    print("AI: " + response.choices[0].message.content)

```

2. 코드를 실행하고 터미널 창에 질문을 입력합니다. 먼저 이름을 알려 주고 다음 질문에서 이름을 물어보면 다음처럼 불과 몇 초 전에 나눈 대화도 기억하지 못합니다.

```

사용자: 안녕? 내 이름은 이성용이야. — 입력
AI: 안녕하세요, 성용님! 만나서 반갑습니다. 오늘 어떻게 도와드릴까요?
사용자: 내 이름이 뭐까?
AI: 당신의 이름은 제가 알 수 없는 정보입니다. 누구를 도와드릴까요?
사용자: exit
PS C:\github\gpt_agent_2024_book>

```

이전 대화 내용 모르는 문제.

3. 이 문제는 사용자의 입력값과 GPT가 생성한 답변을 `messages`에 추가하도록 코드를 작성해 **간단히 해결**할 수 있습니다. 다음 코드를 작성합니다.

```
멀티턴 대화 만들기 multi_turn.py

from openai import OpenAI # 오픈AI 라이브러리 가져오기
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def get_ai_response(messages):
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        temperature=0.9, # 응답 생성에 사용할 temperature 설정
        messages=messages, # 대화 기록을 입력으로 전달
    )
    return response.choices[0].message.content # 생성된 응답의 내용 반환

messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
] # 초기 시스템 메시지 설정

while True:
    user_input = input("사용자: ") # 사용자 입력받기

    if user_input == "exit":
        break

    messages.append({"role": "user", "content": user_input})
    ai_response = get_ai_response(messages) # 대화 기록을 기반으로 AI 응답 가져오기
    messages.append({"role": "assistant", "content": ai_response})
    print("AI: " + ai_response) # AI 응답 출력
```

- ① GPT에서 API로 답변을 받아 오는 부분을 `get_ai_response`라는 함수로 만들고 `messages`를 매개변수로 받게 합니다.
- ② 리스트 형태로 초기 메시지를 설정합니다. `while` 문을 통해 이 리스트에 메시지를 추가할 예정입니다.
- ③ 사용자가 입력한 메시지를 `messages`에 추가합니다.
- ④ `get_ai_response` 함수의 실행 결과를 `messages`에 추가합니다. 이 방식을 통해 GPT와 사용자가 나눈 대화 가 `messages`에 저장된 상태로 대화를 이어 나갈 수 있습니다.

코드를 실행하고 이름을 물어보았습니다. 앞선 예제에서 이름을 GPT에게 말한 뒤 곧바로 물어도 답을 못했지만 이번에는 이름을 기억하고 있습니다. `messages`에 기존 대화 내용이 모두 저장되어 있으므로 적절하게 예측해 대답합니다.

사용자: 안녕? 내 이름은 이성용이야.

AI: 안녕하세요, 이성용님! 만나서 반갑습니다. 오늘 어떻게 도와드릴까요?

사용자: 내가 누구게?

AI: 이성용님에 대해 잘 알지는 못하지만, 친절하고 대화하려는 의지가 있는 분 같아요. 더 알려주시면 제가 더 잘 도와드릴 수 있을 것 같아요. 어떤 이야기를 나누고 싶으신가요?

이번에는 미국에서 인기 있는 연예인은 누구인지 물어본 뒤 ‘한국에서는 어때?’라고만 물어도 앞에서 대화한 내용의 맥락을 파악해서 한국 유명 연예인을 소개해 줍니다.

사용자: 미국에서 인기있는 연예인은 누구야?

AI: 미국에서 인기 있는 연예인은 매우 다양한데요, 최근에는 비욘세, 테일러 스위프트, 드웨인 존슨, 리아나 등이 인기있습니다.

현재 트렌드는 시기에 따라 변화하기 때문에, 최신 정보를 위해 뉴스나 소셜 미디어를 참고하는 것도 좋은 방법입니다. 어떤 특정 분야에 관심이 있으신가요?

사용자: 한국에서는 어때?

AI: 한국에서도 다양한 분야에서 많은 연예인들이 큰 인기를 끌고 있습니다. 최근 주목받는 몇몇 연예인은 방탄소년단, 블랙핑크, 유재석, 아이유 등이 있습니다.

한국은 K-pop, 드라마, 영화 등 여러 분야에서 세계적으로도 큰 영향력을 발휘하고 있어, 연예인들의 인기도 그만큼 높습니다. 특정 분야에 관심이 있으신가요?

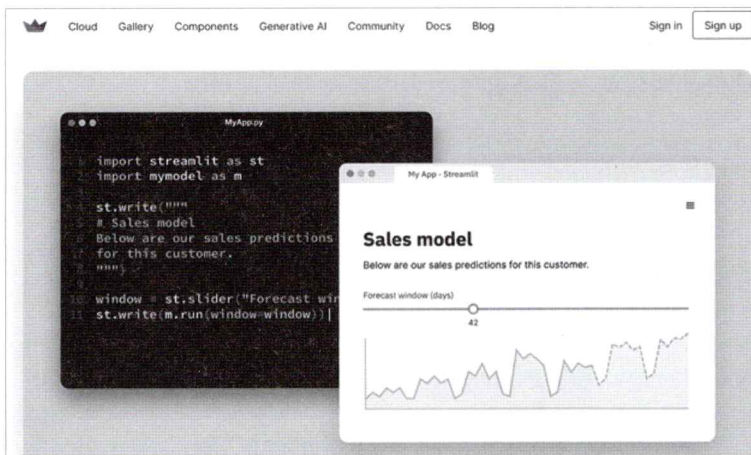
사용자: exit

PS C:\github\gpt_agent_2024_book>

GPT는 항상 입력된 텍스트를 바탕으로 다음에 나올 문장을 예측할 뿐 과거의 기억을 갖고 있지 않습니다. 이 문제는 기존 대화 내용을 계속 쌓아 가면서 해결할 수 있음을 기억해 두세요.

03-3 스트림릿으로 챗봇 완성하기

터미널 창에서 실행한 결과를 깔끔한 사용자 인터페이스 **User Interface, UI**에서 볼 수 있도록 스트림릿 **streamlit**을 사용해 보겠습니다. 스트림릿은 파이썬 프로그램을 웹 기반 UI로 구현할 수 있게 해주는 프레임워크입니다. GPT나 클로드 같은 언어 모델을 사용할 때 채팅 화면을 쉽게 구현할 수 있어서 많은 사람이 스트림릿을 이용합니다.

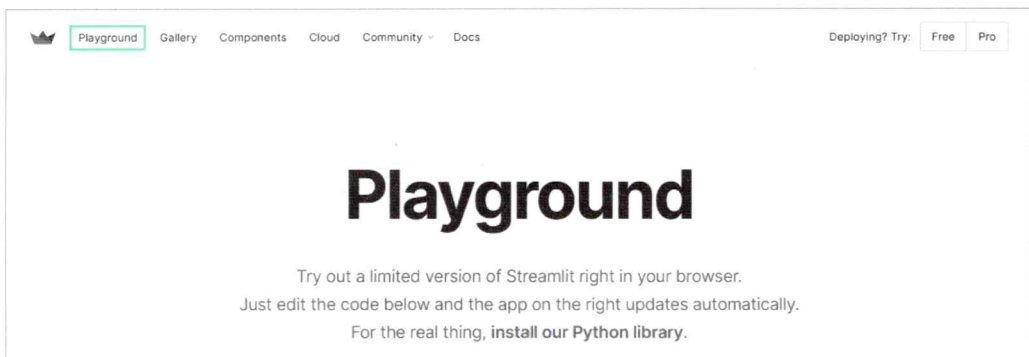


스트림릿 웹 사이트

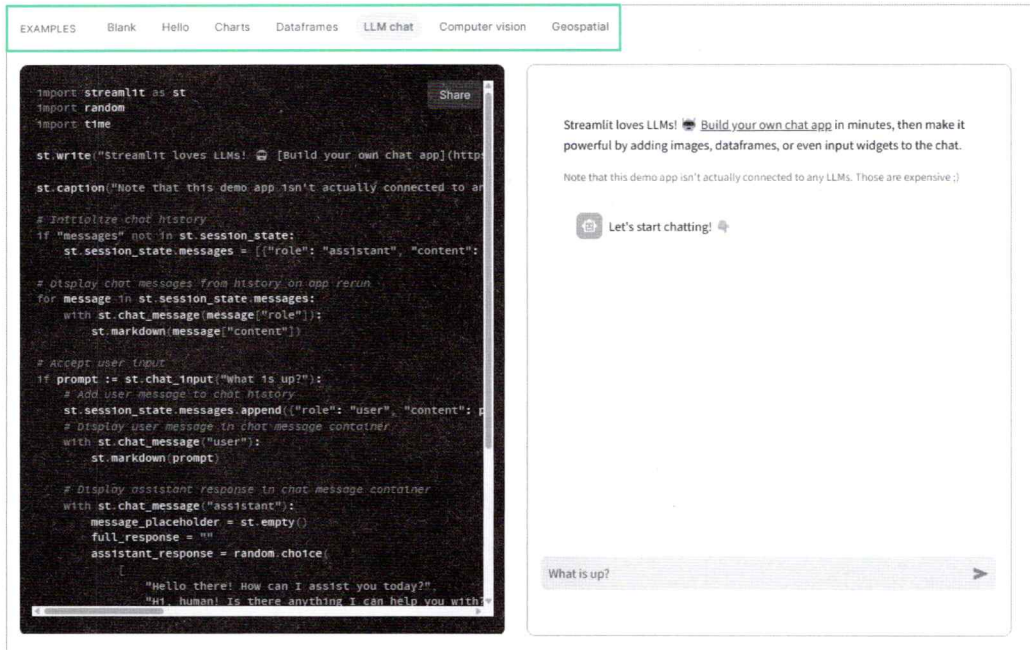
Do it! 실습 스트림릿으로 챗봇 UI 만들기

■ 결과 파일: sec03/streamlit_basic.py

1. 스트림릿 웹 사이트(<https://streamlit.io/>)에 접속하고 첫 화면의 상단 메뉴에서 [Playground]를 선택합니다.



스트림릿에서 제공하는 다양한 예시 코드를 활용하면 쉽게 코드를 작성할 수 있습니다.



2. VS Code 터미널 창에 다음처럼 입력해 스트림릿을 설치합니다. 여기에서는 현재 시점의 최신 버전인 1.41.1을 설치했습니다. 이 책과 동일한 환경에서 실습하고 싶다면 `pip install streamlit==1.41.1`로 설치하세요.

◆ 가상 환경(venv)에서 설치해야 함을 잊지 마세요. 가상 환경 만들기는 02-1절 실습을 참고하세요.

```
(venv) > pip install streamlit
```

3. 새 파일 `streamlit_basic.py`를 만들고 스트림릿 웹 사이트에서 제공하는 코드를 붙여 넣어 일부를 수정합니다. 원래 예시 코드에서는 사용자가 오픈AI의 API 키를 직접 입력하도록 되어 있는데 여기서는 앞서 배운 것처럼 환경 변수에 저장해 둔 API 키를 사용할 수 있게 했습니다. 그리고 모델은 `gpt-4o`로 변경합니다.

◆ 스트림릿에서 제공하는 예시 코드가 변경되었습니다. 이 책과 같은 UI로 실습하고 싶다면 다음 코드를 그대로 작성하세요.

```

import streamlit as st
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

with st.sidebar:
    openai_api_key = os.getenv('OPENAI_API_KEY')
    # openai_api_key = st.text_input("OpenAI API Key", key="chatbot_api_key", type="password")
    "[Get an OpenAI API key](https://platform.openai.com/account/api-keys)"
    "[View the source code](https://github.com/streamlit/llm-examples/blob/main/Chatbot.py)"
    "[![Open in GitHub Codespaces](https://github.com/codespaces/badge.svg)](https://codespaces.new/streamlit/llm-examples?quickstart=1)"

st.title("🗨️ Chatbot")

if "messages" not in st.session_state:
    st.session_state["messages"] = [{"role": "assistant", "content": "How can I help you?"}]

for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"])

if prompt := st.chat_input():
    if not openai_api_key:
        st.info("Please add your OpenAI API key to continue.")
        st.stop()

    client = OpenAI(api_key=openai_api_key)
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)
    response = client.chat.completions.create(model="gpt-4o", messages=st.session_state.messages)
    msg = response.choices[0].message.content
    st.session_state.messages.append({"role": "assistant", "content": msg})
    st.chat_message("assistant").write(msg)

```

- 1 스트림릿은 사이드바에 내용을 추가하는 기능인 `st.sidebar`를 제공합니다. 예시 코드에서는 사용자가 API 키를 입력하도록 되어 있지만 여기서는 코드에 API 키를 넣어서 사용자가 따로 입력할 필요가 없게 수정합니다.
- 2 `st.session_state`는 스트림릿에서 사용자의 세션 상태를 관리하는 기능입니다. 사용자가 웹 브라우저에서 상호 작용하는 동안 그 상태를 유지하고 업데이트할 수 있습니다. `st.session_state`에 `messages`가 없으면 'How can I help you?' 라는 메시지를 초기 응답으로 설정하도록 딕셔너리 형태로 추가되어 있습니다.
- 3 대화 기록을 웹 브라우저에 출력하는 부분입니다. `st.chat_message`는 스트림릿의 채팅 인터페이스에서 메시지를 출력하는 용도로 사용합니다. 각 메시지는 역할이 있으며 여기서는 `assistant`와 `user`입니다. 이 역할을 인자로 받아 해당 역할에 맞는 스타일로 메시지를 표시하고 `.write()`로 화면에 내용을 출력합니다. 이처럼 다양한 요소를 스트림릿에서 제공하여 사용하기 편리합니다.
- 4 사용자의 입력을 받아 `prompt` 변수에 담는 부분입니다. `st.chat_input`은 스트림릿에서 제공하는 기능으로 사용자가 입력한 텍스트를 받아옵니다.
- 5 `openai_api_key`가 정의되지 않았을 때 오류 메시지를 보여주기 위한 부분입니다. 이 코드는 위에서 `os.getenv('OPENAI_API_KEY')`로 정의했으므로 문제는 없습니다.
- 6 사용자가 채팅 창에 질문을 입력하면 해당 내용을 `st.session_state.messages`에 딕셔너리 형태로 추가하고 화면에 사용자 입력 내용을 출력합니다.
- 7 GPT의 답변을 받아 와서 다시 `st.session_state.messages`에 추가하고 답변을 화면에 출력합니다.

4. 터미널 창에 다음과 같이 `streamlit run 파일명.py`을 입력해 스트림릿에서 해당 파일을 실행합니다. 이때 스트림릿의 이메일 뉴스 수신 여부를 묻는 내용이 터미널 창에 출력되는데 무시하고 `[Enter]`를 누르면 진행됩니다.

```
(venv) > streamlit run streamlit_basic.py
```

5. 스트림릿을 처음 사용한다면 보안 설정 창이 표시됩니다. `[허용]`을 클릭하면 진행됩니다.



스트림릿으로 구현한 챗봇 화면입니다. 코드에서 `st.session_state`에 대화 내용을 기록하므로 과거의 대화 내용을 잊지 않고 멀티턴 대화를 할 수 있습니다.

